Documentación Inicial del Proyecto AdoPaws:

Organización del Trabajo y Metodología de Desarrollo

Albany Gonzalez.

José Pablo Monestel.

Avril Madrigal.

Rebeca Arce.

Universidad Invenio

Ingeniería de Software 1

Darin Mauricio Gamboa

25 de marzo de 2026

Resumen

El presente documento describe la organización del equipo de desarrollo de AdoPaws, la metodología ágil aplicada durante el proyecto y la estructura de gestión del trabajo adoptada. Se documenta la implementación de Scrum como marco de trabajo, la distribución de roles del equipo (Product Owner, Scrum Master y Development Team), la organización del trabajo en sprints de dos semanas y el uso de Trello como herramienta de gestión del tablero Kanban. Adicionalmente, se detalla la estrategia de control de versiones con Git y GitHub, la política de ramas y la convención de commits adoptada por el equipo.

Palabras clave: Scrum, metodología ágil, gestión de proyecto, sprints, Trello, GitHub Flow, organización del equipo, control de versiones.

Introducción

El desarrollo exitoso de un proyecto de software no depende únicamente de las decisiones técnicas adoptadas, sino también de la capacidad del equipo para organizarse, comunicarse y gestionar el trabajo de forma estructurada. En el proyecto AdoPaws, la organización del equipo y la metodología de desarrollo fueron definidas desde el inicio como elementos fundamentales para garantizar el avance sostenido del proyecto en el tiempo disponible.

Se adoptó Scrum como marco de trabajo ágil por su idoneidad para proyectos con requisitos que pueden evolucionar durante el desarrollo, su énfasis en la entrega iterativa e incremental de funcionalidades y su estructura de ceremonias que facilitan la inspección y adaptación continua del proceso (Schwaber & Sutherland, 2020). Este documento describe cómo Scrum fue implementado en el contexto específico del proyecto, detallando los roles, artefactos, ceremonias y herramientas de soporte utilizadas.

Metodología de Desarrollo: Scrum

Scrum es un marco de trabajo ágil para el desarrollo de productos complejos que se basa en ciclos cortos de trabajo (sprints), roles claramente definidos y ceremonias de inspección y adaptación frecuentes. Su aplicación en AdoPaws permitió organizar el desarrollo en incrementos funcionales entregables al final de cada sprint, facilitando la detección temprana de desviaciones y su corrección antes de que impactaran el cronograma del proyecto.

Sprints

El proyecto se organizó en sprints de dos semanas de duración, siguiendo la recomendación del Scrum Guide de mantener sprints lo suficientemente cortos para que el equipo pueda inspeccionar el progreso y adaptar el plan con frecuencia (Schwaber & Sutherland, 2020). Cada sprint produjo un incremento funcional del sistema, compuesto por las funcionalidades planificadas en el Sprint Backlog y validadas durante la Sprint Review.

La duración de dos semanas fue seleccionada considerando la naturaleza académica del proyecto y la disponibilidad del equipo. Este ritmo permitió completar ciclos de análisis, diseño, implementación y prueba en cada iteración, asegurando que al final de cada sprint existiera código funcionando que pudiera ser evaluado.

Roles del Equipo

Product Owner

El Product Owner es el responsable de maximizar el valor del producto y gestionar el Product Backlog. En AdoPaws, el Product Owner aprobó la propuesta del proyecto, definió el enfoque general de la plataforma y validó que el sistema desarrollado cumpliera con los objetivos planteados. Aunque su participación activa en el desarrollo diario fue limitada, fue consultado en las Sprint Reviews para evaluar si el incremento producido se alineaba con la visión del producto.

Scrum Master

El Scrum Master es el responsable de garantizar que Scrum sea comprendido y aplicado correctamente por el equipo. En AdoPaws, el Scrum Master coordinó las ceremonias de cada sprint, facilitó la resolución de impedimentos técnicos y organizativos, monitoreó el avance de las tareas en el tablero de Trello y participó activamente en el desarrollo del sistema junto con los demás miembros del equipo. Esta dualidad de rol (Scrum Master y desarrollador) es común en equipos pequeños de proyectos académicos y de startups.

Equipo de Desarrollo

El equipo de desarrollo está conformado por cuatro integrantes que trabajaron de forma colaborativa en todas las capas del sistema. La distribución de tareas no fue fija ni excluyente: cada integrante participó en backend, frontend, base de datos y documentación según las necesidades de cada sprint. Esta flexibilidad facilitó la adaptación ante imprevistos y permitió que

el conocimiento del sistema fuera distribuido entre los miembros del equipo, reduciendo los riesgos asociados a la dependencia de un único desarrollador por área.

Artefactos de Scrum

Product Backlog

El Product Backlog es la lista ordenada de todo el trabajo que podría realizarse en el producto. Para AdoPaws, el Product Backlog fue construido a partir de los 14 requisitos funcionales definidos en el documento de alcance del PoC, expandidos con las funcionalidades adicionales del MVP. Las historias de usuario se redactaron en formato estándar (Como [rol], quiero [funcionalidad], para [beneficio]) y fueron estimadas por el equipo mediante Planning Poker durante las ceremonias de Sprint Planning.

Sprint Backlog

El Sprint Backlog comprende el conjunto de ítems del Product Backlog seleccionados para el sprint en curso, más el plan del equipo para entregar el incremento. En AdoPaws, el Sprint Backlog se materializó en las tarjetas de Trello asignadas en la columna del sprint correspondiente, con responsable, descripción técnica y criterios de aceptación documentados en cada tarjeta.

Incremento

El incremento es la suma de todos los ítems del Product Backlog completados durante el sprint, más los incrementos de todos los sprints anteriores. En AdoPaws, cada incremento correspondió a un conjunto de funcionalidades implementadas, probadas y mergeadas a la rama principal del repositorio. El criterio de Done del equipo establece que una funcionalidad se considera completada cuando el código está implementado, las pruebas unitarias relevantes están escritas, el código ha superado el análisis de SonarQube y ha sido revisado mediante Pull Request por al menos un compañero.

Ceremonias de Scrum

Tabla 5

Ceremonias de Scrum Aplicadas en el Proyecto AdoPaws

Ceremonia	Frecuencia	Duración Aproximada	Propósito
Sprint Planning	Inicio de cada sprint	1-2 horas	Selección de ítems del Product Backlog y planificación del sprint
Daily Scrum	Diaria (días hábiles)	15 minutos	Sincronización del equipo: qué se hizo, qué se hará, impedimentos
Sprint Review	Fin de cada sprint	1 hora	Demostración del incremento y retroalimentación del Product Owner
Sprint Retrospective	Fin de cada sprint	30-45 minutos	Inspección del proceso y definición de mejoras para el siguiente sprint

Nota. Las duraciones son aproximadas y se ajustaron según las necesidades del equipo en cada sprint.

Gestión del Trabajo: Tablero Trello

Trello fue adoptado como herramienta de gestión visual del trabajo por su facilidad de uso, su modelo Kanban basado en tarjetas y columnas y su accesibilidad desde cualquier dispositivo. El tablero del proyecto fue configurado con las siguientes columnas que representan el flujo de trabajo del equipo.

La columna de preparación inicial contiene las tareas necesarias para configurar el entorno de desarrollo: creación del repositorio en GitHub, configuración de la solución .NET, inicialización del proyecto React con Vite, diseño del DER y definición del flujo de trabajo del equipo. La columna de cada sprint contiene las tarjetas planificadas para el ciclo correspondiente, con responsable y criterios de aceptación asignados durante el Sprint Planning. La columna En proceso contiene las tarjetas activamente en desarrollo por algún miembro del equipo. La columna Por revisar contiene las tarjetas completadas que requieren revisión mediante Pull Request antes de ser marcadas como aprobadas. La columna Aprobado contiene las tarjetas revisadas, aprobadas y mergeadas al repositorio.

Cada tarjeta de Trello incluye: título descriptivo de la tarea, descripción técnica con detalles de implementación, miembro del equipo responsable, etiquetas por módulo (Adopciones, Marketplace, Consultas, DevOps, Documentación) y checklist de criterios de aceptación. Este nivel de detalle facilitó la comprensión de cada tarea por cualquier miembro del equipo, independientemente de quién la hubiera creado.

Control de Versiones: Git y GitHub

El control de versiones del proyecto se gestiona con Git como sistema de control de versiones distribuido y GitHub como plataforma de alojamiento remoto del repositorio. La estrategia de branching adoptada es una variante simplificada de GitHub Flow, adecuada para equipos pequeños con ciclos de entrega cortos.

Estrategia de Ramas

El repositorio mantiene dos ramas permanentes: main, que contiene el código en estado estable y deployable, y develop, que integra las funcionalidades completadas en el sprint en curso antes de ser mergeadas a main al final del sprint. Para cada nueva funcionalidad o corrección, el desarrollador crea una rama feature/nombre-funcionalidad o fix/nombre-corrección desde develop, trabaja en ella de forma aislada y abre un Pull Request hacia develop cuando la implementación está completa.

Los Pull Requests requieren la revisión y aprobación de al menos un miembro del equipo antes de poder ser mergeados. Esta práctica garantiza que el código integrado haya sido revisado por al menos dos personas, reduciendo la probabilidad de introducir errores y facilitando la transferencia de conocimiento entre los miembros del equipo. El pipeline de CI/CD se ejecuta automáticamente sobre cada Pull Request, garantizando que el código propuesto compila, pasa las pruebas unitarias y cumple los Quality Gates de SonarQube antes de ser integrado.

Convención de Commits

El equipo adoptó la convención de Conventional Commits para la redacción de mensajes de commit, que establece un formato estructurado compuesto por un tipo, un alcance opcional y una descripción. Los tipos utilizados en el proyecto son: feat para nuevas funcionalidades, fix para corrección de errores, docs para cambios en documentación, test para adición o modificación de pruebas, refactor para cambios que no añaden funcionalidades ni corrigen errores, y chore para tareas de mantenimiento como actualizaciones de dependencias o configuración del pipeline.

Esta convención facilita la generación automática de changelogs, la comprensión del historial de cambios del proyecto y la identificación rápida del tipo de modificación introducida por cada commit. Ejemplos de mensajes de commit válidos según esta convención serían: feat(pets): add photo upload endpoint, fix(auth): correct JWT token expiration, test(compatibility): add unit tests for rule-based scoring, o ci: configure GitHub Actions workflow for .NET backend.

Tabla 6

Estructura del Repositorio GitHub del Proyecto AdoPaws

Directorio / Archivo	Contenido
Adopaws/	Solución .NET con los proyectos Adopaws.Api, Adopaws.Application, Adopaws.Domain y Adopaws.Infrastructure
adopaws-frontend/	Proyecto React 18 + Vite con la SPA del frontend

.github/workflows/ci.yml	Archivo de configuración del pipeline CI/CD con GitHub Actions
sonar-project.properties	Configuración del proyecto en SonarQube/SonarCloud
README.md	Documentación de instalación y ejecución del proyecto

Nota. El repositorio aloja ambos proyectos (backend y frontend) en un monorepo, facilitando la gestión unificada del pipeline CI/CD.

Comunicación del Equipo

La comunicación del equipo se articuló mediante tres canales complementarios. El canal de comunicación informal y síncrona se utilizó para la coordinación diaria y la resolución rápida de dudas durante el desarrollo. Trello funcionó como canal asíncrono de gestión de tareas, donde cada miembro podía consultar en cualquier momento el estado del trabajo y actualizar el avance de sus tareas. GitHub constituyó el canal de revisión técnica: los Pull Requests y sus comentarios sirvieron como espacio de discusión sobre decisiones de implementación, mejoras de código y corrección de errores antes de la integración.

La combinación de estos tres canales garantizó que la información relevante para el desarrollo fuera accesible, trazable y persistente, reduciendo la dependencia de la comunicación informal no documentada que suele generar malentendidos y pérdida de contexto en equipos de desarrollo.

Conclusión

La organización del trabajo adoptada en AdoPaws demostró ser adecuada para el contexto del proyecto. La implementación de Scrum con sprints de dos semanas, combinada con el uso de Trello para la gestión visual del trabajo y GitHub para el control de versiones y la revisión de código, proporcionó al equipo la estructura necesaria para avanzar de forma ordenada y sostenida en el desarrollo del sistema.

La distribución flexible de tareas entre los miembros del equipo, la práctica de Pull Requests con revisión cruzada y la convención de commits estructurados contribuyeron a la calidad del código producido y al conocimiento compartido del sistema. Estos elementos, en conjunto con el ciclo DevOps implementado (pruebas automatizadas, pipeline CI/CD, inspección con SonarQube), establecieron las bases para un desarrollo profesional y mantenible de la plataforma AdoPaws.

Referencias

- Atlassian. (2024). *Trello: Manage your team's projects from anywhere*. https://trello.com/

- Conventional Commits. (2024). *A specification for adding human and machine readable meaning to commit messages*. https://www.conventionalcommits.org/

- GitHub. (2024). *GitHub Flow: A lightweight branch-based workflow*. https://docs.github.com/en/get-started/using-github/github-flow

- Pressman, R. S., & Maxim, B. R. (2015). *Ingeniería del software: Un enfoque práctico* (8.ª ed.). McGraw-Hill Education.

- Rubin, K. S. (2012). *Essential Scrum: A practical guide to the most popular agile process*. Addison-Wesley.

- Schwaber, K., & Sutherland, J. (2020). *The Scrum guide: The definitive guide to Scrum – The rules of the game*. Scrum.org. https://scrumguides.org/